

ZAŁOŻENIA PROJEKTU

Porównanie modeli Machine Learning

Od czystych danych, przez pełne porównanie algorytmów, po optymalizację hiperparametrów zwycięzcy.

1. Cel projektu

Celem projektu jest przeprowadzenie **pełnego, systematycznego procesu budowy modelu predykcyjnego** — od przygotowania czystych danych, przez wytrenowanie i uczciwe porównanie wielu różnych algorytmów, aż po optymalizację hiperparametrów najlepszego z nich. Projekt ma pokazać, że potraficie samodzielnie przejść całą ścieżkę poznaną na zajęciach — nie tylko wytrenować pojedynczy model, ale **uzasadnić**, dlaczego wybrany model jest najlepszym wyborem dla danego problemu.

Projekt realizujecie indywidualnie, na własnym, samodzielnie wybranym zbiorze danych (zgodnie z kryteriami z sekcji 2).

2. Wymagania dotyczące danych

Dane muszą być czyste — to znaczy, że przed etapem modelowania musicie wykonać (i udokumentować) pełny proces czyszczenia danych:

- Braki danych (missing values) — zidentyfikowane i świadomie obsłużone (usunięcie, imputacja medianą/modą/inną metodą — z uzasadnieniem wyboru)
- Duplikaty — sprawdzone i usunięte, jeśli występują
- Zmienne kategoryczne — zakodowane (One-Hot Encoding / Label Encoding, w zależności od charakteru zmiennej)
- Outliers — zidentyfikowane; decyzja o ich zachowaniu lub usunięciu musi być świadoma i uzasadniona, nie automatyczna
- Standaryzacja/normalizacja cech numerycznych — wykonana tam, gdzie tego wymaga algorytm (kNN, SVM, MLP, regresja logistyczna)
- Finalny zbiór — zero braków danych, wyłącznie wartości numeryczne, gotowy do bezpośredniego podania do `.fit()`

Kryteria doboru datasetu: minimum 300 obserwacji, minimum 5 cech, jasno zdefiniowany target (klasyfikacja binarna, wieloklasowa lub regresja). Dataset nie może być identyczny z żadnym użytym na zajęciach (Iris, Breast Cancer, Titanic, Sonar, Phoneme, Wine Quality, Ionosphere itd.) — poszukajcie czegoś świeżego, najlepiej z dziedziny, która Was interesuje.

3. Zakres modeli do przetestowania

Musicie przetestować **wszystkie** poniższe modele na tym samym, jednolicie przygotowanym podziale train/test — to jest warunek uczciwego porównania. Dla każdego modelu obowiązkowa jest walidacja

krzyżowa (min. 5-fold) na zbiorze treningowym.

Kategoria	Modele	Uwaga
Baseline liniowy	Logistic Regression / Linear Regression	Punkt odniesienia — jeśli złożone modele go nie biją, to sygnał ostrzegawczy
Sąsiedzi i margines	kNN, SVM	Wymagają standaryzacji; dobierz k / kernel systematycznie
Drzewa i ensemble	Decision Tree, Random Forest, Gradient Boosting	Random Forest/GB nie wymagają standaryzacji
Boosting (biblioteki)	XGBoost, LightGBM (CatBoost — opcjonalnie)	Zwykle wygrywają ze sklearnowym Gradient Boosting
Sieć neuronowa	MLPClassifier / MLPRegressor	Standaryzacja obowiązkowa; przetestuj min. 3 różne architektury

Minimum 8 modeli ocenionych w jednym, zbiorczym porównaniu (wykres słupkowy lub tabela: model × accuracy/RMSE × czas treningu). Traktujcie to zestawienie jako kluczowy artefakt projektu — to on odpowiada na pytanie „który model wybrać i dlaczego”.

4. Metodologia oceny

- 1. Podział danych** — train/test (80/20 lub 70/30) ze stratify dla klasyfikacji. Zbiór testowy dotykany TYLKO RAZ, na samym końcu.
- 2. Walidacja krzyżowa** — min. 5-fold cross-validation na zbiorze treningowym — do porównania modeli i doboru hiperparametrów. Nigdy na zbiorze testowym.
- 3. Metryki** — klasyfikacja: accuracy + minimum jedna z (precision, recall, F1, AUC) — szczególnie ważne przy niezbalansowanych klasach. Regresja: RMSE + MAE + R^2 .
- 4. Standaryzacja w Pipeline** — użyj `make_pipeline(StandardScaler(), model)`, żeby standaryzacja odbywała się osobno w każdym foldzie CV — bez przecieku danych (data leakage).

5. Optymalizacja hiperparametrów

Po wyłonieniu najlepszego modelu z porównania (sekcja 3) przeprowadźcie jego **systematyczny tuning**:

- Zdefiniuj siatkę/przestrzeń hiperparametrów — minimum 3 hiperparametry, każdy z minimum 3 wartościami do sprawdzenia
- Użyj `GridSearchCV` (mała siatka) lub `RandomizedSearchCV` (duża/ciągła przestrzeń, zalecane przy >30 kombinacjach)
- Porównaj model dostrojony z domyślnym — pokaż konkretną liczbową poprawę (albo uczciwie napisz, że tuning nie dał istotnej poprawy, jeśli tak było)
- Dla modeli sieciowych (MLP): przetestuj wpływ `alpha` (regularyzacja L2) i rozważ `early_stopping=True`

6. Finalny wybór i uzasadnienie

Projekt musi zawierać jawne podsumowanie: **który model wybieracie jako finalny, i dlaczego** — nie tylko na podstawie najwyższej accuracy, ale z uwzględnieniem:

- Kompromisu jakość / czas treningu / interpretowalność
- Stabilności wyniku (odchylenie standardowe accuracy w CV, nie tylko średnia)
- Ryzyka przeuczenia (różnica train vs test accuracy)
- Aspektu Green IT — czy prostszy model dawałby niemal tak samo dobry wynik znacznie mniejszym kosztem obliczeniowym?

Uwaga: model z najwyższą accuracy NIE zawsze jest właściwym wyborem. Doceniamy świadome uzasadnienie kompromisu bardziej niż mechaniczny wybór najwyższej liczby w tabeli.

7. Struktura projektu (deliverables)

Projekt oddajecie w standardowym repozytorium kursu, w swoim folderze `projekt/literaimienia_nazwisko/` (ten sam workflow fork → commit → PR, o którym mówiliśmy przy okazji prac domowych).

```
projekt/literaimienia_nazwisko/
├─ README.md           opis datasetu, źródło, cel biznesowy
├─ 01_czyszczenie.ipynb wczytanie + pełne czyszczenie danych
├─ 02_eda.ipynb         eksploracja, wizualizacje, korelacje
├─ 03_porownanie_modeli.ipynb min. 8 modeli, jedno zbiorcze porównanie
├─ 04_tuning_finalny.ipynb optymalizacja hiperparametrów + wybór
└─ dane/               surowe i/lub oczyszczone dane (CSV)
```

Alternatywnie: cały projekt w jednym, dobrze zorganizowanym notebooku z czytelnymi sekcjami Markdown — jeśli wolicie taką formę, upewnijcie się, że sekcje odpowiadają strukturze z tabeli powyżej i są jasno oddzielone nagłówkami.

8. Kryteria oceny

Obszar	Waga	Na co zwracam uwagę
Czystość danych	20%	Brak NaN w finalnym zbiorze, udokumentowane i uzasadnione decyzje (nie tylko „dropna() na wszystkim”)
Kompletność porównania	25%	Wszystkie wymagane modele obecne, ten sam podział danych, uczciwa metodologia (brak przecieku danych)
Optymalizacja hiperparametrów	20%	Sensowna siatka parametrów, poprawne użycie CV, pokazana różnica przed/po
Uzasadnienie wyboru modelu	20%	Nie tylko liczba — argumentacja uwzględniająca kompromisy z sekcji 6

Obszar	Waga	Na co zwracam uwagę
Jakość kodu i czytelność	15%	Nazwy zmiennych, komentarze, wykresy z opisanymi osiami, README wyjaśniający kontekst datasetu

Skala ocen zgodna z ogólną skalą kursu ($\geq 90\%$ → 5.0, z proporcjonalnym przełożeniem niżej) — szczegóły jak przy pracach domowych.

9. Sugerowany harmonogram

Etap	Zakres	Sugerowany czas
1	Wybór datasetu + eksploracja wstępna	1 dzień
2	Czyszczenie danych (01_czyszczenie.ipynb)	1-2 dni
3	EDA — wizualizacje, korelacje, rozkłady (02_eda.ipynb)	1 dzień
4	Trening i porównanie min. 8 modeli (03_porownanie_modeli.ipynb)	2-3 dni
5	Tuning hiperparametrów + finalne uzasadnienie (04_tuning_finalny.ipynb)	1-2 dni
6	Przegląd, porządkowanie kodu, README	1 dzień

Pytania dotyczące doboru datasetu lub interpretacji wymagań — zgłaszajcie na bieżąco, najlepiej przed rozpoczęciem etapu 2.